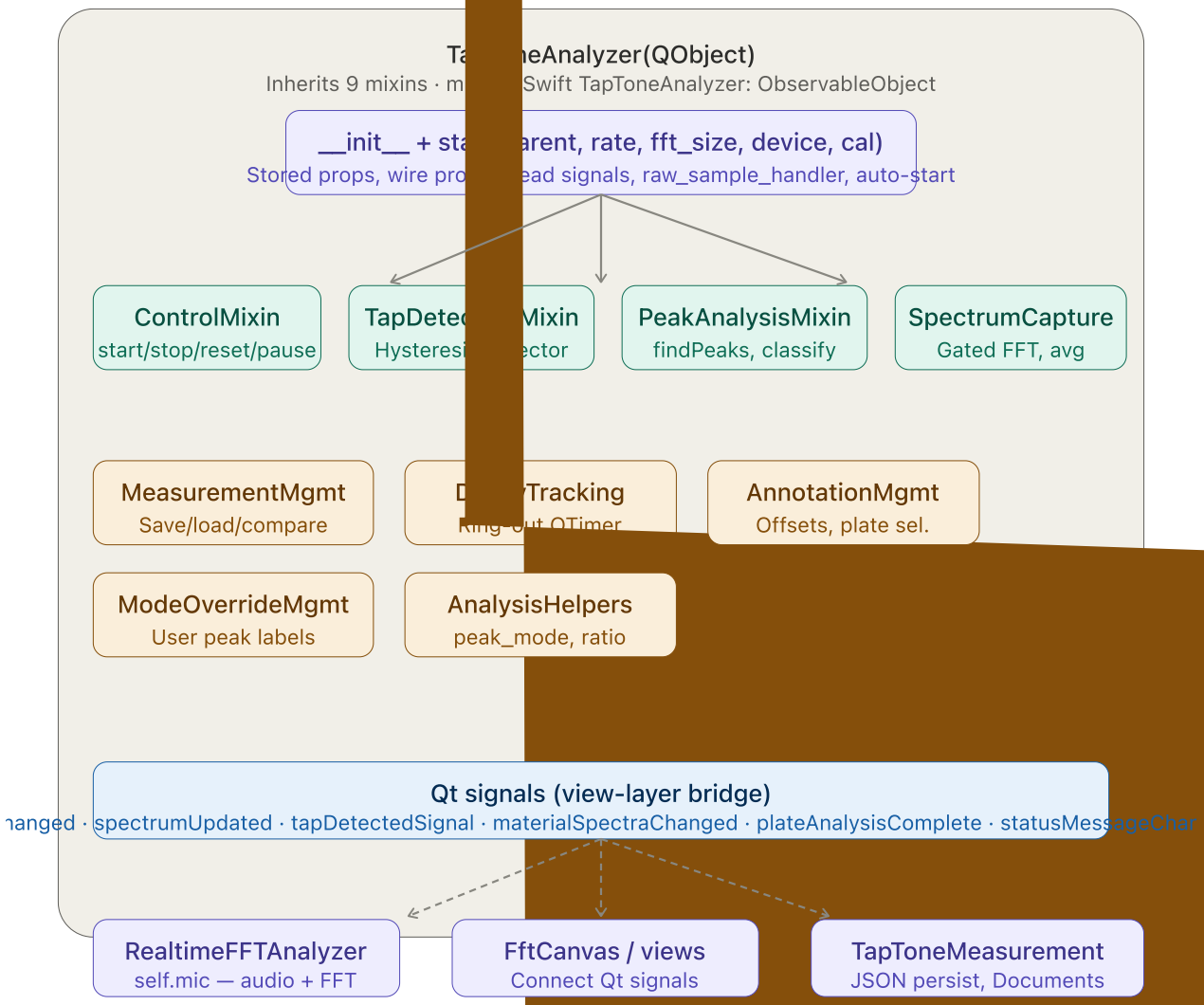
GuitarTap Python — TapToneAnalyzer Architecture

tap_tone_analyzer.py · 9 mixin modules · mirrors Swift TapToneAnalyzer + 8 extensions

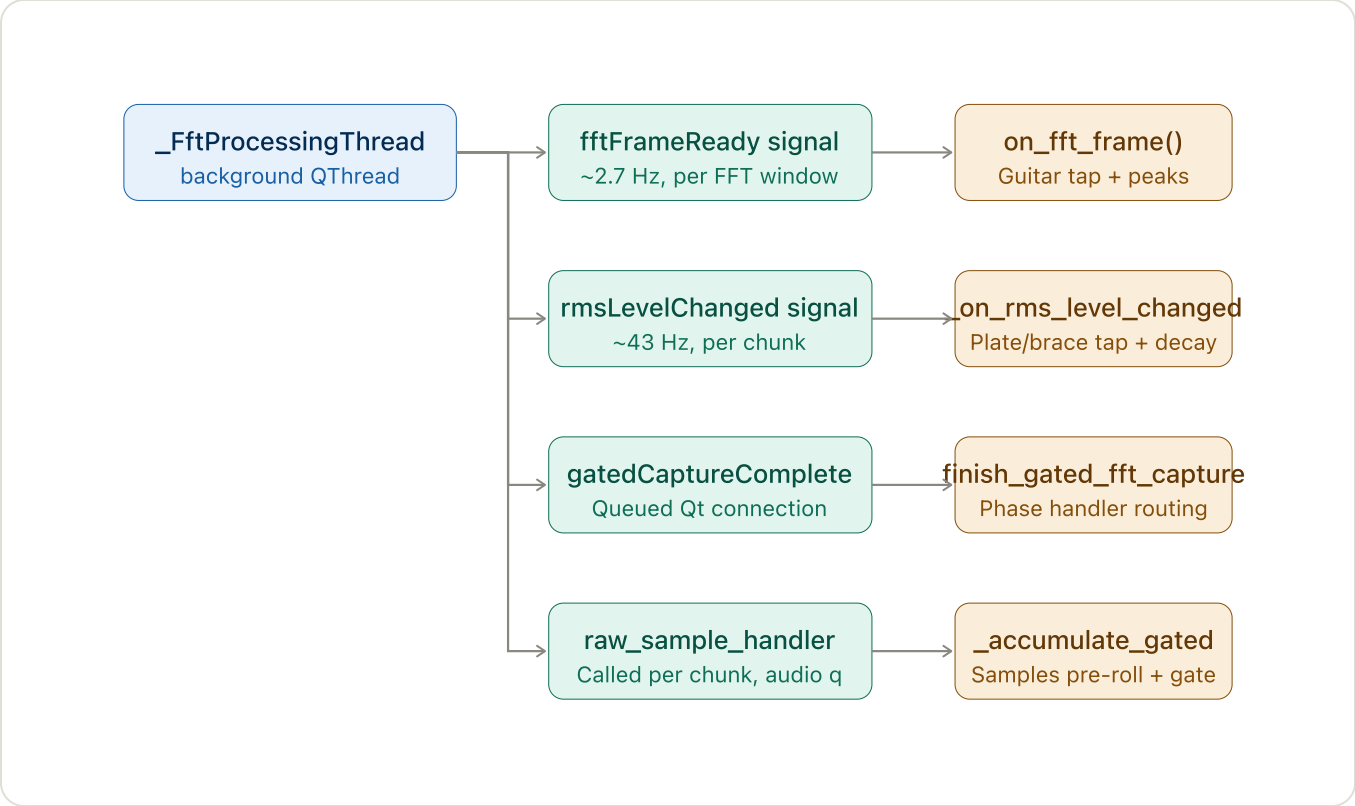
1. Full class architecture

`TapToneAnalyzer` inherits from 9 mixins plus `QObject`. The base file holds all stored properties, `__init__`, Qt signals, the `start()` wiring method, and a handful of helper methods that Swift places directly on the class (not in any extension). Unlike Swift's `@Published` Combine-driven UI, Python uses Qt signals to notify the view layer.



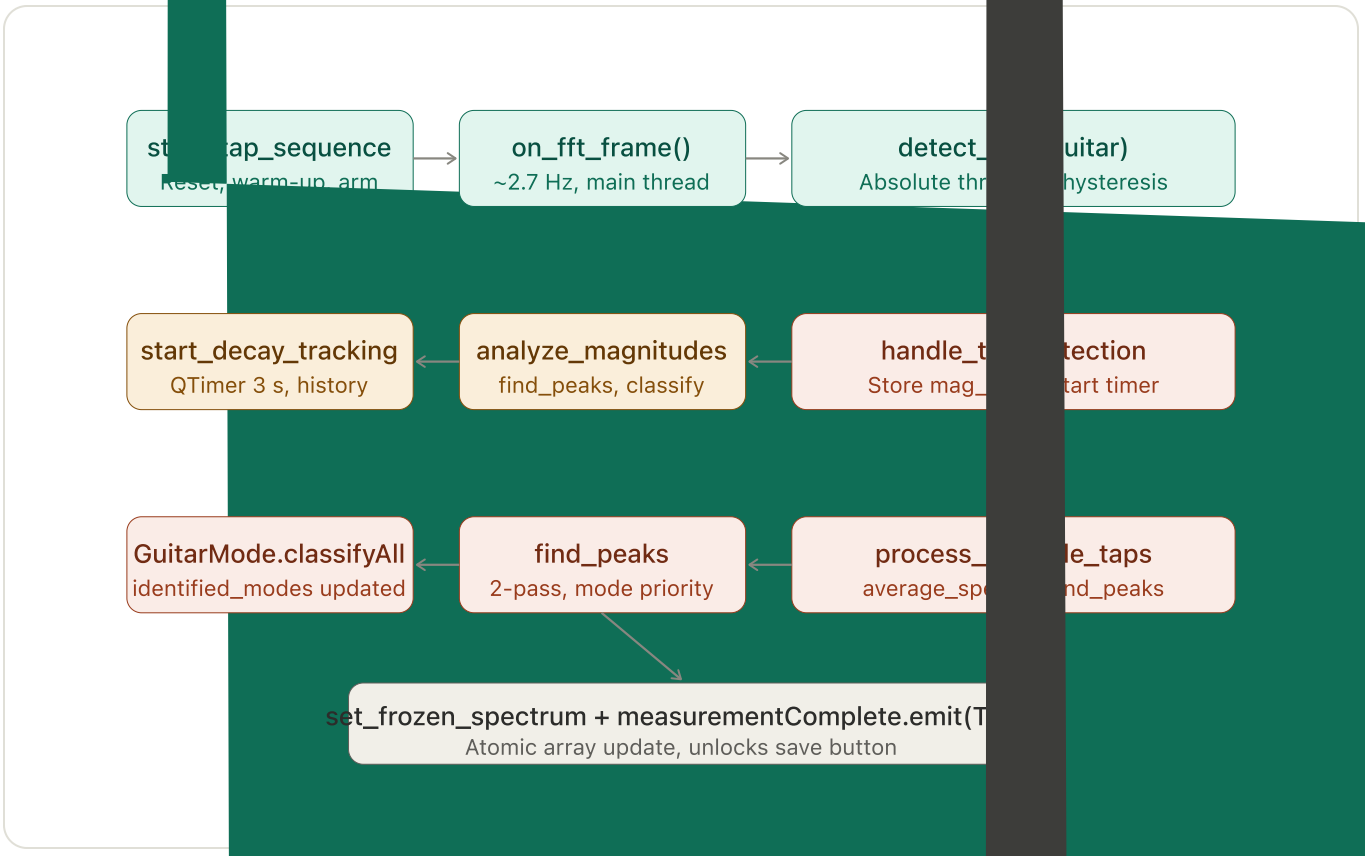
2. FFT frame wiring and signal flow

Python replaces Swift's Combine subscriptions with Qt signal connections wired in `start()`. `fftFrameReady` (per-FFT, ~2.7 Hz) drives `on_fft_frame` which handles guitar-mode tap detection and peak analysis. `rmsLevelChanged` (~43 Hz) drives `_on_rms_level_changed` for plate/brace tap detection and decay tracking. `gatedCaptureComplete` delivers gated-FFT results to `finish_gated_fft_capture` on the main thread.



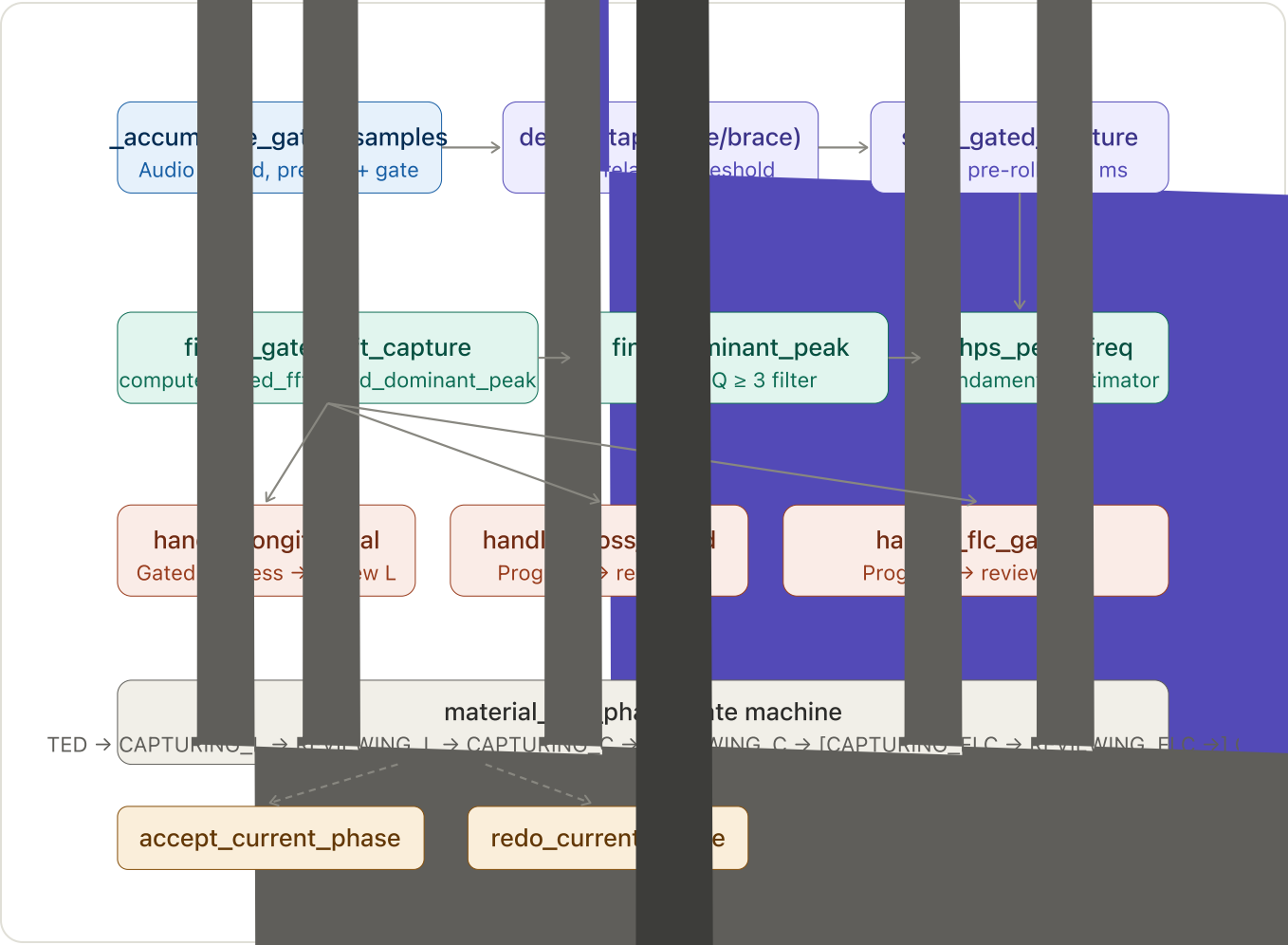
3. Guitar mode measurement flow

`on_fft_frame` is the main entry point for guitar mode. It calls `detect_tap` (absolute threshold hysteresis) and `analyze_magnitudes` on every FFT frame. On a rising edge, `handle_tap_detection` stores the spectrum, starts decay tracking, and schedules `process_multiple_taps` after the capture window. Spectrum averaging runs in the linear power domain.



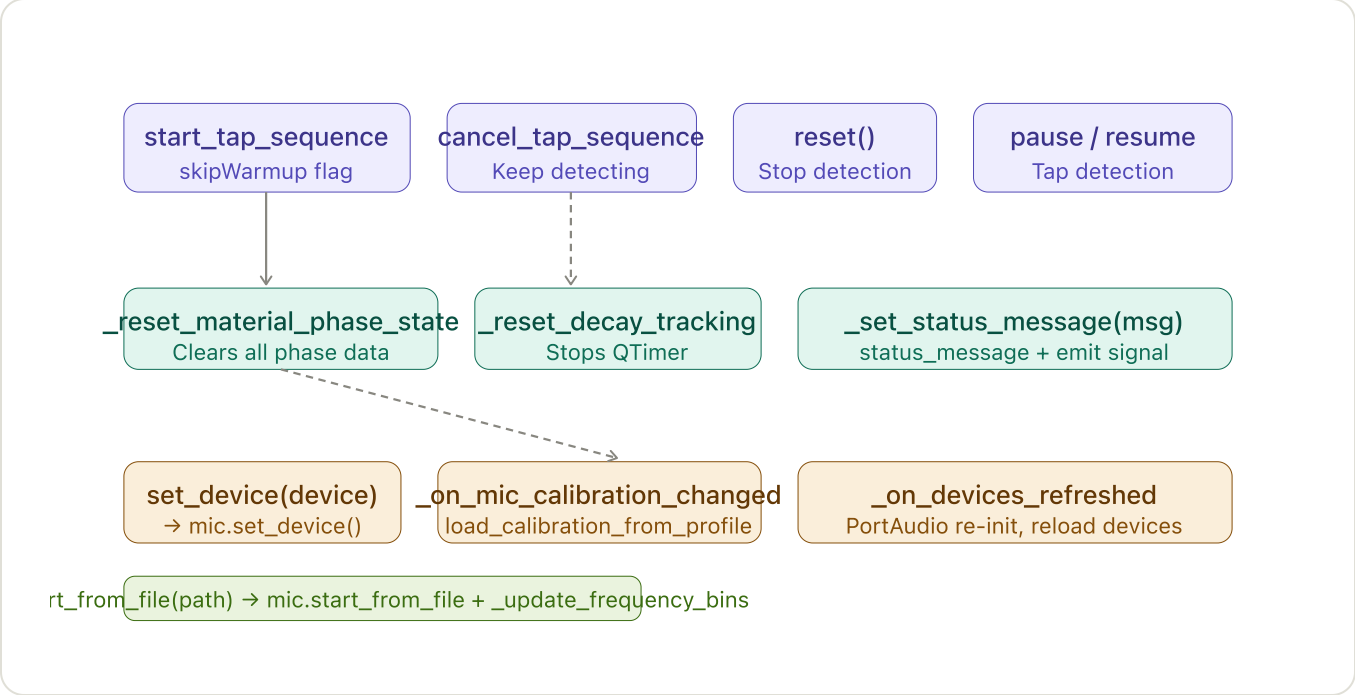
4. Plate / brace gated FFT flow

Plate and brace modes bypass the continuous FFT path entirely. `_accumu gated_samples` (called via `raw_sample_handler` on the audio thread) maintains the 200 e-roll ring buffer and accumulates the 400 ms gate window. When the window fills, it emits `ga ptureComplete` (queued Qt connection) delivering the samples to `finish_gated_fft_ca` on the main thread. The phase-state machine progresses through L → C → FLC with Accept/Redo at each review step.



5. Control, calibration, and device manage t

`ControlMixin` owns the lifecycle: `start_tap_se e`, `cancel_tap_sequence`, `reset`, `pause/resume_tap_detection`, and the Accept/Redo phase transitions. It also owns calibration loading and device switching. Hot-plug events arrive on `_devicesRefreshed` signal, triggering PortAudio re-init and `mic.load_available_input_devices()`



Key architecture notes

Swift @Published → Python Qt signals

Every Swift `@Published` property that drives UI has a corresponding Qt signal emitted explicitly from code. For example, writing `status_message` always goes through `_set_status_message()` which emits `statusMessageChanged`, mirroring Swift's automatic `objectWillChange` trigger on every assignment. The `measurementComplete`, `tapCountChanged`, `plateAnalysisComplete`, and `materialSpectraChanged` signals map to Swift's frozen-spectrum and isMeasurementComplete publishers.

Gated capture ownership

Pre-roll buffer (`_pre_roll_buf`), gated accumulator (`_gated_accum`), `_gated_capture_active`, and `_gated_lock` live on `TapToneAnalyzer`, not on `_FftProcessingThread`. This matches Swift where `TapToneAnalyzer.accumulateGatedSamples(_:sampleRate:)` owns the buffers. `_accumulate_gated_samples` is set as `mic.raw_sample_handler` so it is called by the processing thread on every audio chunk. When the window fills, it emits `gatedCaptureComplete` (a queued Qt signal on the processing thread) which delivers the samples to `finish_gated_fft_capture` on the main thread.

Tap detection mode split

Mode	Signal	Rate	Threshold
Guitar	`fftFrameReady` → `on_fft_frame`	~2.7 Hz	Absolute dBFS

Plate / Brace	`rmsLevelChanged` → `_on_rms_level_changed`	43 Hz	Relative: noise floor EMA + headro

Key Python-only differences from Swift

- `_set_status_message(msg)` — explicit helper for all s string writes, `statusMessageChanged` . Swift uses `@Published va atusMessage w`
- `recreate_proc_thread()` — destroys and replaces ProcessingThread equivalent reset via `AVAudioEngine.stop()` / `star`
- `_on_devices_refreshed()` — calls `sd._terminate` `sd._initialize()` enumerating. Swift CoreAudio doesn't cache the dev t.
- `_update_frequency_bins()` — called after file play changes `mic.rate` `updateFrequencyBins()` in `startFromFile` .
- Three-layer peak selection (`effective_longitudina` `ak_id` etc.) us with `or` -chain logic, exactly mirroring Swift's `??` ch
- `set_frozen_spectrum` uses Python's GIL as the ato guarantee (no `objectWillChange.send()` needed) — both arrays e before any vi are on the main thread.

Measurement persistence differences

Python delegates to `views.tap_analysis_results_vie` `ve_all_measurements` `load_all_measurements` rather than owning the file URL itself (Swift's `measurementsFileURL` static property on `TapToneAnalyzer`). The test-path redirect by the view layer in Python.

Generated by Claude from GuitarTap Python source analysis · tap_tone_analyzer.py + 9 mixin modules